

OWASP Juice Shop — Web Security Assignment

Secure Feature Design · Login Form Implementation · Exploitation Testing

Student: Siva Sai Deepank Manoj

Email: deeps45@tamu.edu

Public GitHub repository: <https://cursor.com/codebase/siva-sai-deepank-manoj/juice-guard>

*Assignment requires a **public** repository. Open the repo page above, go to settings, and set visibility from Private to **Public** before you submit.*

Part 1 — Secure Feature Design

While exploring OWASP Juice Shop, three realistic attack paths stood out and shaped a secure registration/login design.

(1) SQL Injection on login. Juice Shop's login historically accepts crafted email input that can manipulate a concatenated SQL query and bypass authentication (classic authentication bypass via injection). **Mitigation:** never build SQL from raw strings — use parameterized queries or an ORM lookup keyed by email. Reject malformed emails before they reach the data layer. This prevents the attacker's payload from changing query logic.

(2) Cross-Site Scripting (XSS). Search boxes and user-controlled fields (reviews, names) can reflect or store HTML/JavaScript that runs in another user's browser, stealing sessions or defacing pages. **Mitigation:** encode all untrusted output for the correct context, prefer `textContent` over `innerHTML`, set a Content-Security-Policy, and reject angle brackets / event-handler patterns on input. Encoding ensures scripts are displayed as text, not executed.

(3) Authentication bypass / weak token handling. Predictable or poorly verified session tokens let attackers impersonate administrators. **Mitigation:** issue signed tokens with strong secrets, short expiry, and server-side authorization checks on every sensitive route. Combined with rate limiting and generic login errors (no user enumeration), forged or replayed credentials are far harder to abuse.

Secure password handling (bcrypt example)

Passwords must never be stored in plaintext. At registration, hash once with a high bcrypt cost factor; at login, compare with `bcrypt.compare`:

```
const passwordHash = await bcrypt.hash(password, 12);
// store passwordHash only – never the plaintext

const match = await bcrypt.compare(submittedPassword, user.passwordHash);
if (!match) return res.status(401).json({ error: 'Invalid email or password.' });
```

Bcrypt embeds a salt and is deliberately slow, which resists rainbow tables and offline brute force. Even if the database leaks, attackers obtain hashes, not reusable passwords.

Part 2 — Simple Front-End Login Form

I implemented a Juice Shop-style login page using plain HTML, CSS, and JavaScript on the client, plus a small Express server for real server-side validation and bcrypt password verification.

What the form includes

- Email and password inputs with labels and accessible status messaging.
- **Client-side validation** (`public/app.js`): blocks empty submissions; requires the email to contain @; requires password length ≥ 8 .
- **Server-side validation** (`server.js`): re-checks email format, password length, and unsafe characters; looks up users without string-built SQL; verifies passwords with `bcryptjs` (cost 12); returns generic errors; rate-limits login attempts.
- Status text is set with `textContent` to avoid DOM XSS.

How to run

```
npm install
npm start
# open http://127.0.0.1:3847
# demo account: demo@juice.shop / JuiceShop1!
```

GitHub repository (must be public)

<https://cursor.com/codebase/siva-sai-deepank-manoj/juice-guard>

The repository README explains the project purpose, structure, `bcrypt` example, and run instructions. Source files: `public/index.html`, `public/app.js`, `server.js`.

The image shows a web application interface for a secure login form. At the top, it says "OWASP JUICE SHOP" in small yellow letters. Below that, the heading "Sign in securely" is displayed in a large, bold, white font. Underneath the heading, a subtitle reads: "Demo login with client-side checks and server-side validation — built for a web security assignment." The main part of the form is a light gray box with a white background. It contains two input fields: "Email" with the placeholder text "you@example.com" and "Password" with the placeholder text "At least 8 characters". Below these fields is a yellow "Log in" button. At the bottom of the form box, there is a line of text: "Demo account: demo@juice.shop / JuiceShop1!". The entire form is centered on a dark blue background.

Figure 1. Secure login form UI (Part 2 implementation).

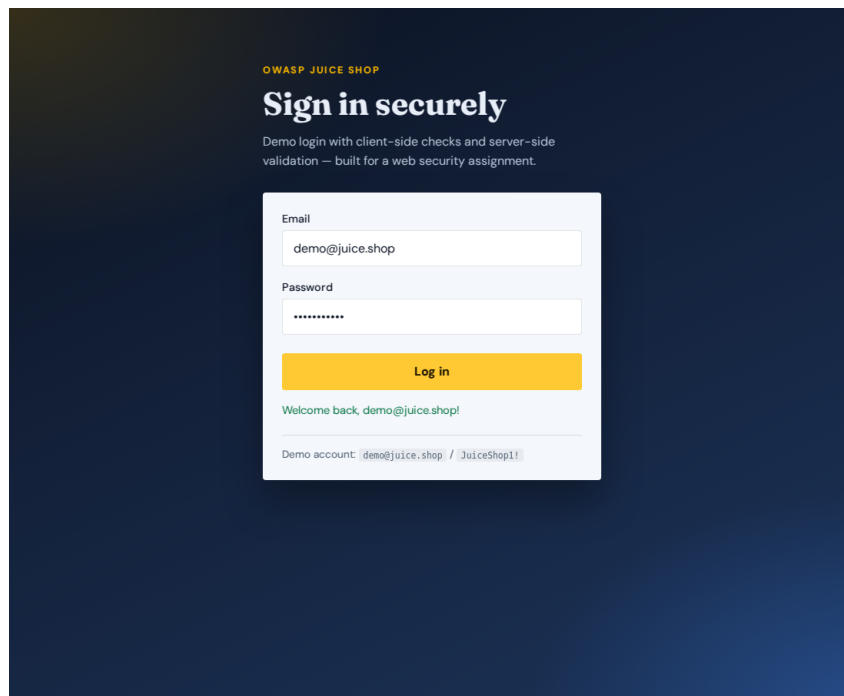


Figure 2. Successful login with demo credentials after client + server checks.

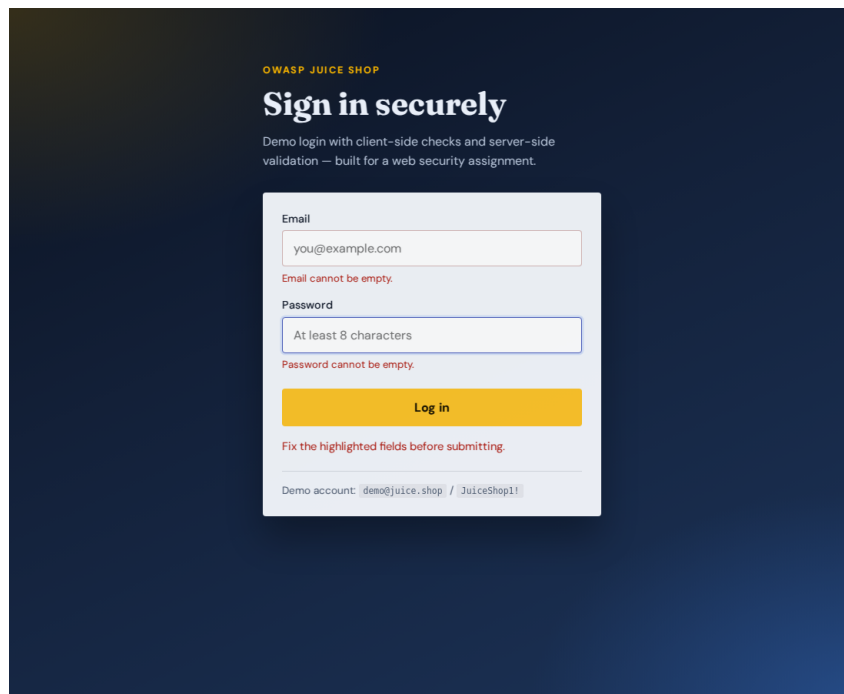


Figure 3. Client-side validation blocking empty email/password submission.

OWASP JUICE SHOP

Sign in securely

Demo login with client-side checks and server-side validation — built for a web security assignment.

Email

Password

Password must be at least 8 characters.

Log in

Fix the highlighted fields before submitting.

Demo account: `demo@juice.shop` / `JuiceShop1!`

Figure 4. Client-side check requiring passwords of at least 8 characters.

Part 3 — Exploitation Attempts Against My Form

I attempted common login attacks against my own form to verify the defenses. Both attempts were blocked — evidence that validation and safe rendering are working.

Attempt A — Cross-Site Scripting (XSS)

Steps: (1) Open the login page. (2) Enter email `xss@test.com`. (3) Enter password `<script>alert(1)</script>`. (4) Click Log in. (5) Observe whether a JavaScript alert appears or script executes.

Result: The attack **did not succeed**. No alert dialog opened. The server rejected the password because it contains unsafe characters (`<` `>`). The UI also renders messages with `textContent`, so even reflected strings would not execute as HTML.

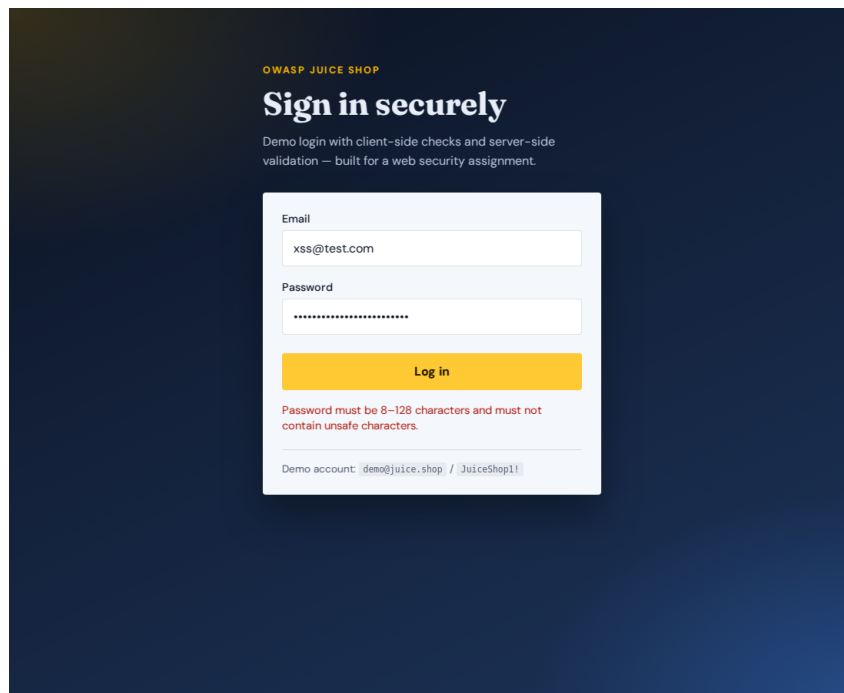


Figure 5. XSS payload rejected — server error: unsafe characters in password.

Attempt B — SQL Injection

Steps: (1) Reload the login page. (2) Enter email `admin@test.com' OR '1'='1`. (3) Enter password `password1`. (4) Click Log in. (5) Check whether authentication was bypassed or data leaked.

Result: The attack **did not succeed**. The application rejected the malformed email and never concatenates user input into SQL (users are stored in an in-memory Map looked up by normalized email). No admin access and no credential dump occurred.

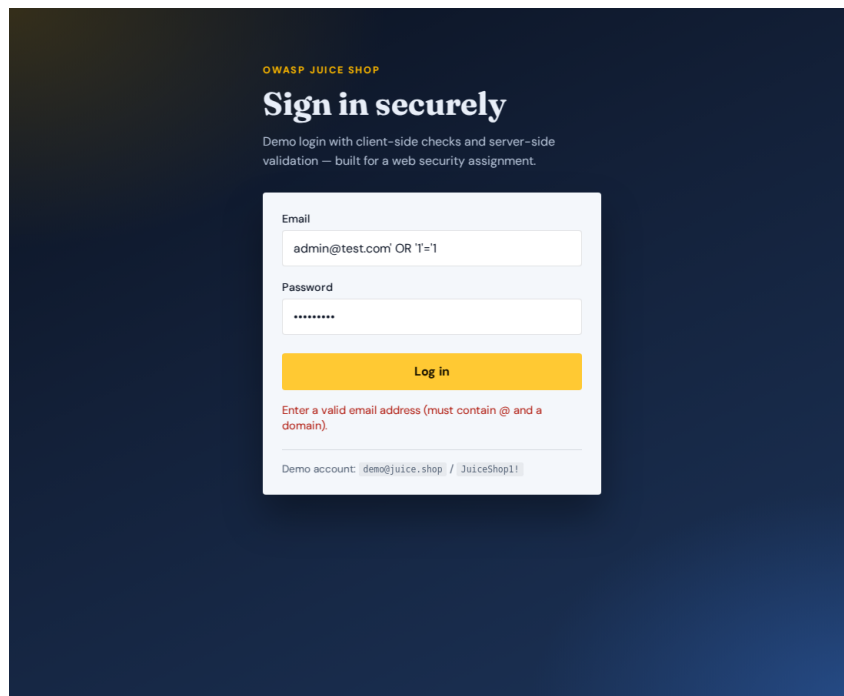


Figure 6. SQL injection–style email rejected as an invalid email address.

Suggested hardening fix

Even though exploitation failed, I also applied a strict **Content-Security-Policy** response header (default-src 'self'; script-src 'self', with limited exceptions for Google Fonts). CSP provides defense in depth: if a future change accidentally uses `innerHTML`, the browser still blocks inline script execution. Additional headers: `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`.

Learning: Client checks improve UX, but server validation, bcrypt hashing, safe DOM updates, and avoiding dynamic SQL are what actually stop SQLi and XSS. Failing to break my own form is a positive outcome for secure design.

Appendix — Repository checklist for graders

- Public GitHub repo with README (what it does + how to run)
- HTML/JS login form with email + password fields
- Client validation: non-empty, email contains @, password ≥ 8 chars
- Server validation + bcrypt password hashing
- Part 1–3 write-ups with screenshots of blocked XSS/SQLi attempts